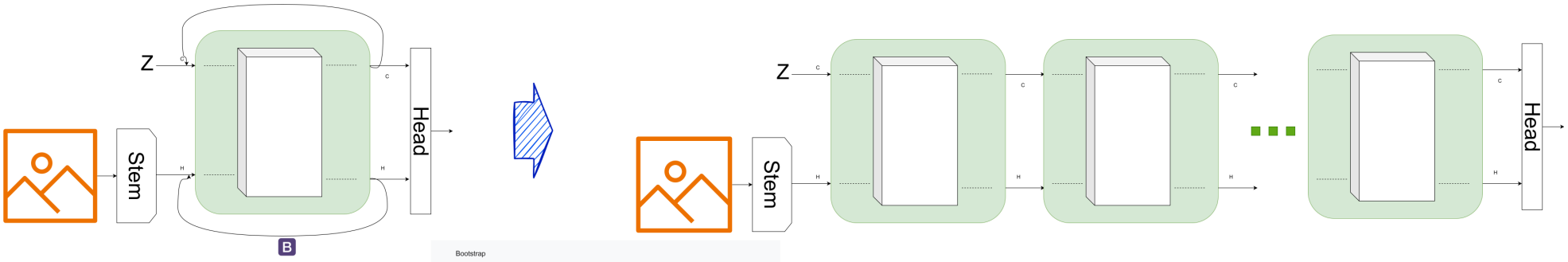
GRLNet-StabhRec: общая концепция рекуррентной развертки



Общая идея. Однослойная модель *GRLNet-StabhRec* не содержит 12 различных сверточных блоков по глубине. Вместо этого она использует один *stem*, один блок инициализации скрытого состояния h-*seed*, одну общую рекуррентную ячейку и одну основную классификационную голову. Рекуррентная ячейка применяется 12 раз подряд к скрытому состоянию фиксированного размера, поэтому эффективная глубина сети возникает за счет временной развертки, а не за счет стека разных слоев с независимыми параметрами.

На концептуальной диаграмме верхний поток помечен маленькой буквой c и соответствует каналу памяти C. Обозначение Z на входе указывает на нулевую начальную инициализацию этого канала перед началом рекуррентной развертки.

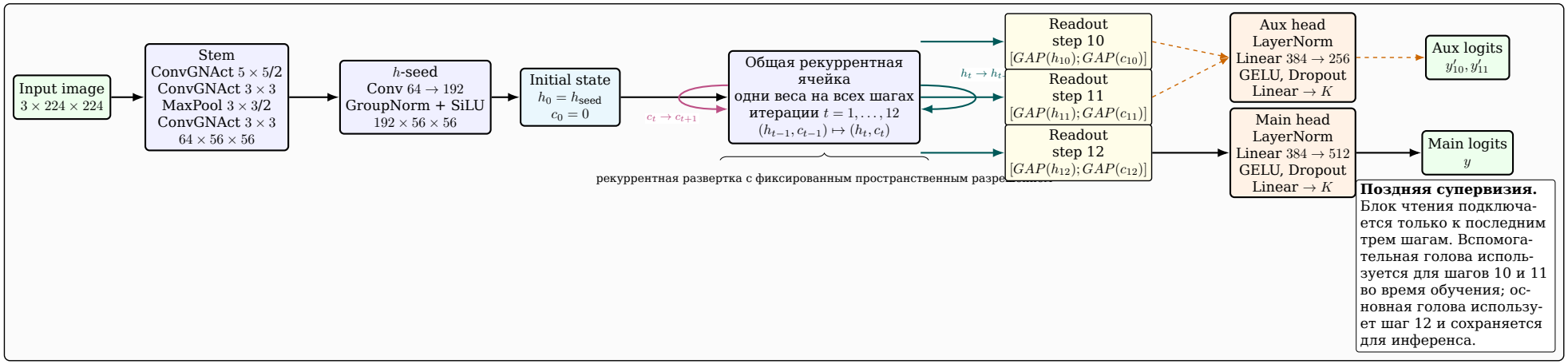
Если считать только уникальные trainable Conv/Linear-слои, то при инференсе модель содержит 9 слоев: 3 слоя в *stem*, 1 слой в h-*seed*, 3 сверточных слоя в рекуррентной ячейке и 2 линейных слоя в основной голове. Во время обучения дополнительно присутствует *aux-head*, что дает еще 2 линейных слоя и увеличивает число уникальных trainable-слоев до 11.

Если же рассматривать сеть в полностью развернутом по времени виде и считать каждое применение общей ячейки как отдельный вычислительный этап, то эффективная глубина возрастает. При инференсе получаем:

$$3\ (\textit{stem}) + 1\ (h\text{-seed}) + 12 \times 3\ (\textit{cell}) + 2\ (\textit{main head}) = 42$$

последовательных Conv/Linear-преобразования. Таким образом, *GRLNet-StabhRec* можно интерпретировать как компактную архитектуру с небольшим числом уникальных параметризованных слоев, но с существенно большей эффективной глубиной в развернутом режиме.

GRLNet-StabhRec: single-layer recurrent architecture



Текстовое описание общей схемы. Входное изображение размера $3 \times 224 \times 224$ сначала обрабатывается блоком *stem*, который уменьшает пространственное разрешение до 56×56 и формирует карту признаков размера $64 \times 56 \times 56$. Затем блок h-seed отображает этот тензор в скрытое рекуррентное пространство, создавая начальное состояние $h_0 \in \mathbb{R}^{192 \times 56 \times 56}$, тогда как состояние памяти инициализируется как $c_0 = 0$.

Ядром модели является одна общая рекуррентная ячейка, применяемая в течение 12 последовательных шагов. Одни и те же параметры ячейки используются на каждом шаге, поэтому сеть увеличивает эффективную глубину за счет итеративного уточнения, а не за счет стека различных слоев. В течение всей рекуррентной развертки и скрытое состояние, и состояние памяти остаются на одной и той же пространственной сетке.

Предсказания снимаются не с каждого рекуррентного шага. Вместо этого блоки readout подключаются только к последним трем шагам, а именно к шагам 10, 11 и 12. На каждом из этих шагов модель выполняет глобальный average pooling как для h_t, так и для c_t, конкатенирует полученные векторы и использует итоговое представление как вход классификатора.

Финальная предиктовая голова работает с readout шага 12 и формирует logits для инференса. Во время обучения дополнительная auxiliary-head применяется также к readout шагов 10 и 11, обеспечивая позднюю дополнительную супервизию. Таким образом, при инференсе используется только последнее рекуррентное состояние, тогда как обучение получает дополнительный сигнал ошибки на двух предшествующих поздних шагах.

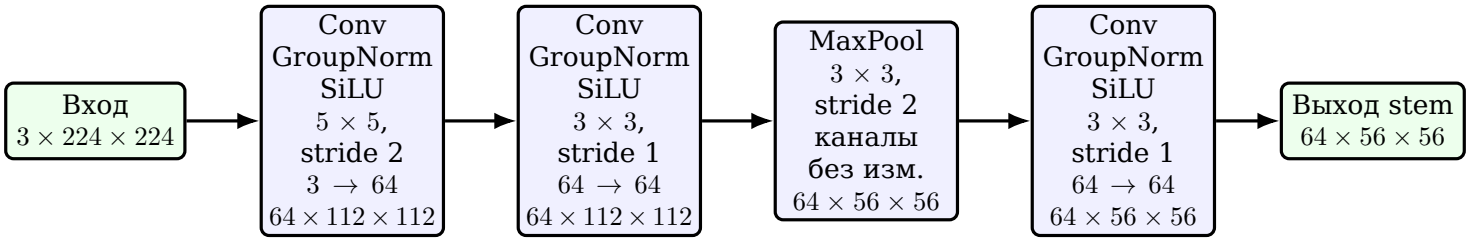
Ключевые элементы рецепта обучения. Для однослойной модели на *ImageNet-1K* использовался оптимизатор `SGD + Nesterov momentum` с линейным warmup на первых эпохах и последующим *cosine decay*. Основной loss строился как `CrossEntropy` с `label smoothing`, а к шагам 10 и 11 дополнительно подключался auxiliary-loss, так что обучение происходило не только по последнему шагу развертки, но и по двум поздним промежуточным readout.

Существенной частью рецепта является *MixUp*: на стадии обучения входные батчи с некоторой вероятностью заменяются линейной смесью двух изображений, а loss считается как та же выпуклая комбинация двух target-функций. Поэтому train-метрики вычисляются в более жестком режиме, чем clean validation. Кроме того, после каждого шага оптимизации обновляется *EMA*-копия весов модели; именно эта сглаженная по времени версия затем используется для валидации, выбора лучших checkpoint

и итогового инференса. Это важно учитывать при интерпретации различий между train- и val-кривыми.

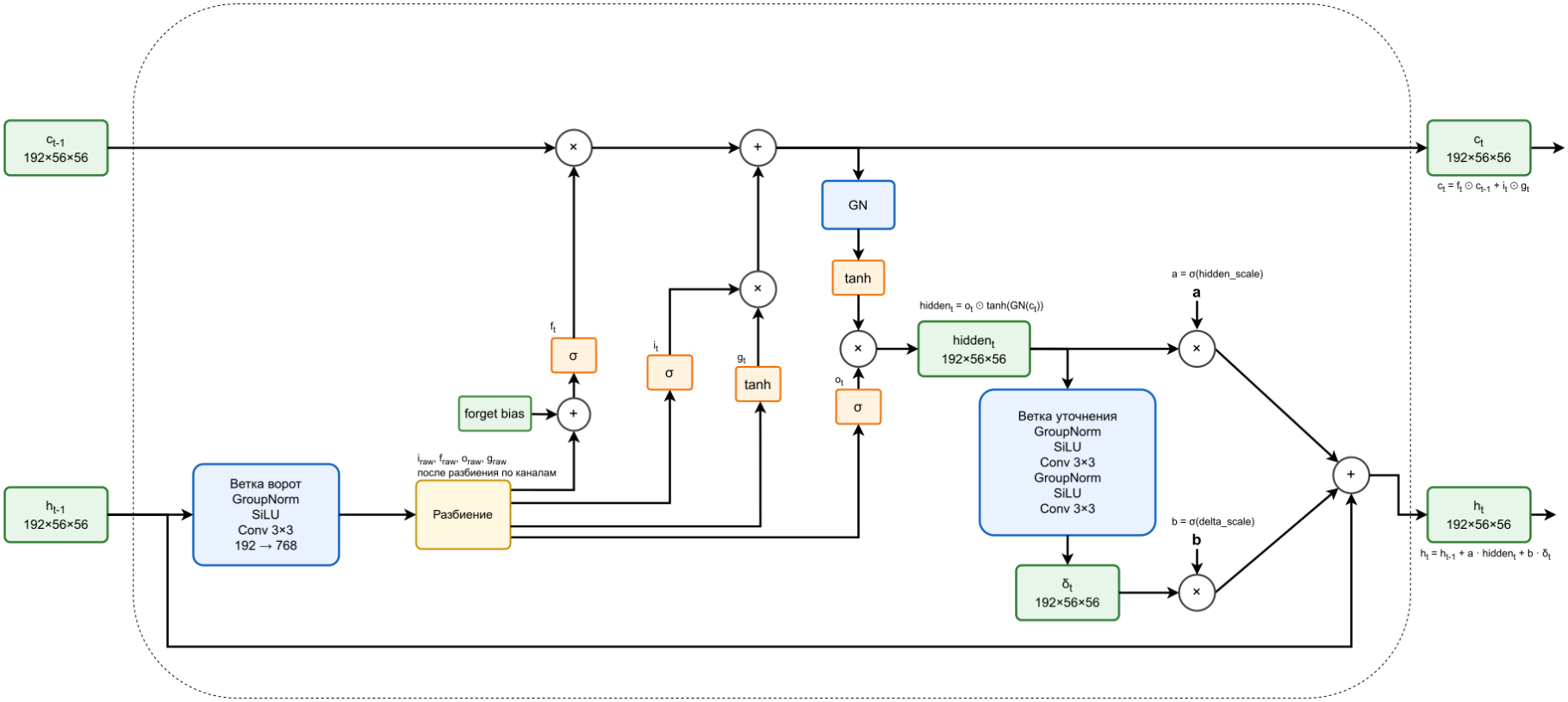
На уровне вычислительного recipe обучение выполнялось в mixed-precision режиме с дополнительным *gradient clipping*; для ускорения использовались `channels_last` и аппаратные оптимизации уровня *A100*. Эти детали не меняют формулу модели, но входят в фактический тренировочный протокол и поэтому важны для корректного воспроизведения результатов.

Отдельная схема Stem.



Краткое описание блока Stem. Первый сверточный слой с ядром 5×5 и шагом 2 выполняет начальное извлечение локальных признаков и одновременно уменьшает пространственное разрешение входа. Второй сверточный слой 3×3 уточняет полученные низкоуровневые признаки без изменения числа каналов и пространственного масштаба. Затем операция max-pooling дополнительно уменьшает разрешение, сохраняя наиболее выраженные локальные ответы. Последний сверточный слой 3×3 формирует итоговое представление блока *stem* размера $64 \times 56 \times 56$, которое затем подается на вход блока h-seed для инициализации рекуррентного скрытого состояния.

Отдельная схема рекуррентной ячейки.



Подробное описание этапов обработки в рекуррентной ячейке. На вход ячейки поступают два состояния предыдущего шага: скрытое состояние h_{t-1} и состояние памяти c_{t-1}. Сначала из h_{t-1} через ветку *gate branch* последовательно вычисляются нормализованные и нелинейно преобразованные признаки, после чего свертка формирует тензор из четырех групп каналов. Этот тензор разбивается на сырые карты i_{raw}, f_{raw}, o_{raw} и g_{raw}, соответствующие входным воротам, воротам забывания, выходным воротам и кандидату обновления памяти.

Далее к ветви забывания добавляется константный *forget bias*, после чего применяются нелинейности: сигмоида для i_t, f_t и o_t, а гиперболический тангенс для g_t. Состояние памяти обновляется по LSTM-подобному правилу

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t,$$

где одна часть старой памяти сохраняется через f_t, а новая информация записывается через i_t и g_t.

После этого новое состояние памяти c_t нормализуется, проходит через функцию tanh и затем поэлементно умножается на выходные ворота o_t. Так формируется основной кандидат обновления скрытого состояния

$$hidden_t = o_t \odot \tanh(GN(c_t)).$$

Этот сигнал отражает основной результат текущего рекуррентного шага.

Затем $hidden_t$ подается в дополнительную ветку *delta / refine*, состоящую из двух последовательных блоков *GroupNorm* $\rightarrow$ *SiLU* $\rightarrow$ *Conv*. Эта ветка вычисляет корректирующую добавку δ_t, предназначенную для более тонкого нелинейного уточнения скрытого состояния. Таким образом, основная ветвь задает базовое gated-обновление, а refine-ветка формирует дополнительную поправку к нему.

На последнем этапе старое скрытое состояние h_{t-1} передается по residual-связи напрямую к выходу ячейки. К нему добавляются два новых вклада: основной сигнал $hidden_t$ и корректирующий сигнал δ_t, каждый со своим обучаемым скалярным коэффициентом:

$$h_t = h_{t-1} + a \cdot hidden_t + b \cdot \delta_t, \qquad a = \sigma(hidden_scale), \quad b = \sigma(delta_scale).$$

Коэффициенты a и b являются обучаемыми глобальными весами, которые регулируют, насколько сильно в итоговое скрытое состояние должны входить основной gated-update и refine-поправка. В результате выходами ячейки становятся обновленные состояния c_t и h_t, сохраняющие ту же пространственную структуру, что и на входе.

Численная проверка вклада компонентов ячейки. Для проверки того, что ветви ячейки не являются декоративными, была проведена inference-only диагностика на первых 256 изображениях из *ImageNet-1K val* с использованием лучшего EMA-checkpoint однослойной модели (`epoch 49`, `best val top-1 = 67.91%`). На этом подмножестве базовая модель дает `67.58%` *top-1*. Затем из forward-pass поочередно удалялись отдельные компоненты ячейки.

Вариант	**Top-1, %**	**Δ к baseline, п.п.**	**Интерпретация**
Базовая модель	67.58	0.00	Все ветви активны
Без C в readout	4.69	-62.89	C несет критическую информацию в классификатор
Без переноса памяти $f_t \odot c_{t-1}$	25.00	-42.58	Канал C работает как память, а не только как градиентный обход
Без вклада $a \cdot hidden_t$	64.06	-3.52	Основной gated-update информативен, хотя не доминирует
Без вклада $b \cdot \delta_t$	3.13	-64.45	Refine-ветка критична для качества
Занулены conv-веса в gate/refine	0.39	-67.19	Один residual-путь по H почти не работает без рекуррентных сверток
Без residual-связи по h_{t-1}	0.00	-67.58	Residual-ветвь необходима для устойчивого накопления состояния

Эти абляции показывают, что в текущей версии архитектуры канал C уже нельзя интерпретировать только как вспомогательный путь для протечки градиента. Напротив, он выполняет две содержательные функции: во-первых, аккумулирует gated-memory через член $f_t \odot c_{t-1}$; во-вторых, напрямую участвует в readout, и удаление pooled-C из классификатора почти разрушает качество. Таким образом, residual-связь по H действительно берет на себя роль стабильной магистрали распространения сигнала, а C смещается в сторону памяти и пространственно согласованного накопления признаков.

Параметры смешивания после обучения принимают значения $a = \sigma(hidden_scale) = 0.0674$ и $b = \sigma(delta_scale) = 0.0255$. Несмотря на то, что $b < a$, фактический вклад refine-ветки оказывается выше из-за большой нормы δ_t. На шагах 1, 6 и 12 средние RMS-вклады равны: для $a \cdot hidden_t$ — `0.0115 / 0.0074 / 0.0051`, а для $b \cdot \delta_t$ — `0.0321 / 0.0301 / 0.0296`. Это означает, что δ-ветка не просто слегка корректирует hidden-update, а фактически выступает главным механизмом уточнения состояния на поздних шагах. При этом вклад $a \cdot hidden_t$ остается ненулевым и его удаление все же снижает top-1 на `3.52` п.п., поэтому базовый gated-update также информативен.

Отдельная абляция с занулением всех рекуррентных сверток внутри ячейки — как в gate-ветке, так и в refine-ветке — дополнительно показывает, что residual-связь по H не шунтирует рекуррентную обработку. В этом режиме сеть фактически сводится

к *stem* $\rightarrow$ *h_seed* $\rightarrow$ многократному тождественному переносу H $\rightarrow$ *readout/head*, однако top-1 падает до `0.39%`. Следовательно, высокая точность базовой модели не может объясняться одним лишь глубоким residual-путем: рекуррентные сверточные преобразования внутри ячейки действительно выполняют основную содержательную работу.

Важно, однако, что приведенная абляция относится только к стадии инференса: обученная модель тестируется с искусственно отключенным членом $a \cdot hidden_t$. Поэтому эти числа не исключают, что данный член играет более существенную роль именно во время оптимизации, стабилизируя ранние шаги обучения и помогая сформировать ту динамику, в которой refine-ветка δ_t затем берет на себя основную часть позднего уточнения.

Численная диагностика протечки градиента. Чтобы уточнить роль каналов H и C в текущей архитектуре, была дополнительно измерена обратная протечка градиента через рекуррентную развертку. Для этого на первых 128 изображениях из *ImageNet-1K val* вычислялся backward по loss, соответствующему финальной стадии обучения:

$$\mathcal{L} = \mathcal{L}_{main} + 0.05 \cdot \mathcal{L}_{aux},$$

где использовалась `CrossEntropyLoss` с `label smoothing = 0.05`, а auxiliary loss подключался к шагам 10 и 11 так же, как в основном рецепте обучения.

Величина	**Шаг 1**	**Шаг 6**	**Шаг 12**
$\|\partial\mathcal{L}/\partial h_t\|_{RMS}$	0.001103	0.000368	0.000024
$\|\partial\mathcal{L}/\partial c_t\|_{RMS}$	0.000089	0.000034	0.000106
$\|\partial\mathcal{L}/\partial h_t\|/\|\partial\mathcal{L}/\partial c_t\|$	12.51	10.95	0.24

Полученная картина показывает, что исходный тезис о канале C как основном носителе глубокого градиента уже не соответствует новой архитектуре. На ранних и средних шагах рекуррентности градиент по каналу H оказывается примерно в 11–13 раз сильнее, чем по каналу C, что хорошо согласуется с появлением прямой residual-связи

$$h_t = h_{t-1} + a \cdot hidden_t + b \cdot \delta_t.$$

Именно эта связь превращает H в основную магистраль переноса градиента через глубокую развертку.

В то же время канал C не становится декоративным. Во-первых, его градиент остается ненулевым на всех шагах. Во-вторых, на последнем шаге градиент по C даже превосходит градиент по H, поскольку pooled-C напрямую входит в readout и влияет на классификатор. Таким образом, в текущей модели C естественнее интерпретировать не как главный обходной путь для градиента, а как memory-ветвь, которая аккумулирует gated-state, участвует в позднем readout и добавляет собственный обучающий сигнал в окрестности выходных шагов.